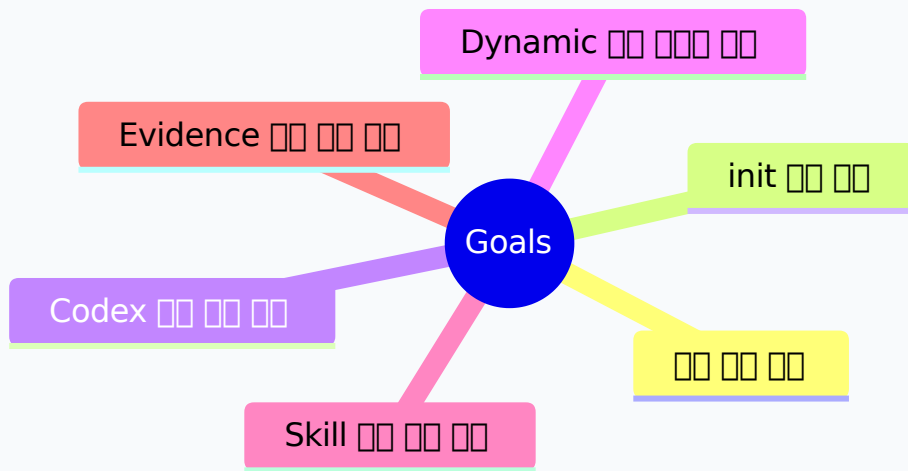


Codex와 Android Deeplink Audit 사용자 가이드

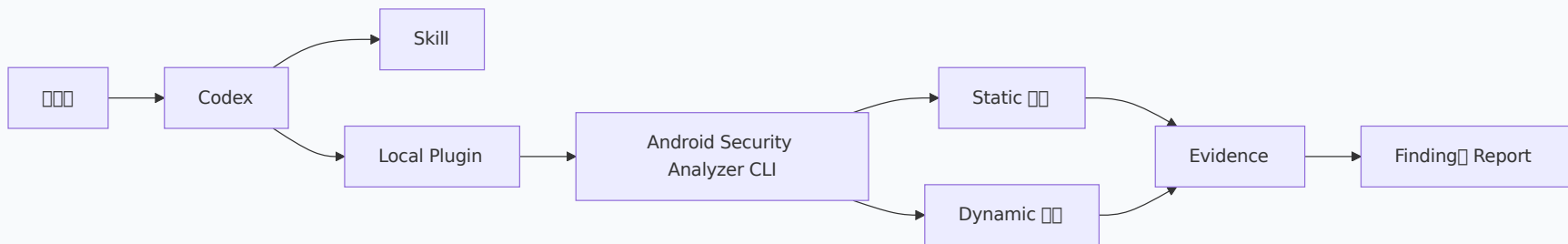
설치부터 활용까지 한 번에 보기

CLI 설치, init 셋업, Codex 활용 흐름, 산출물 확인 방법

Goals

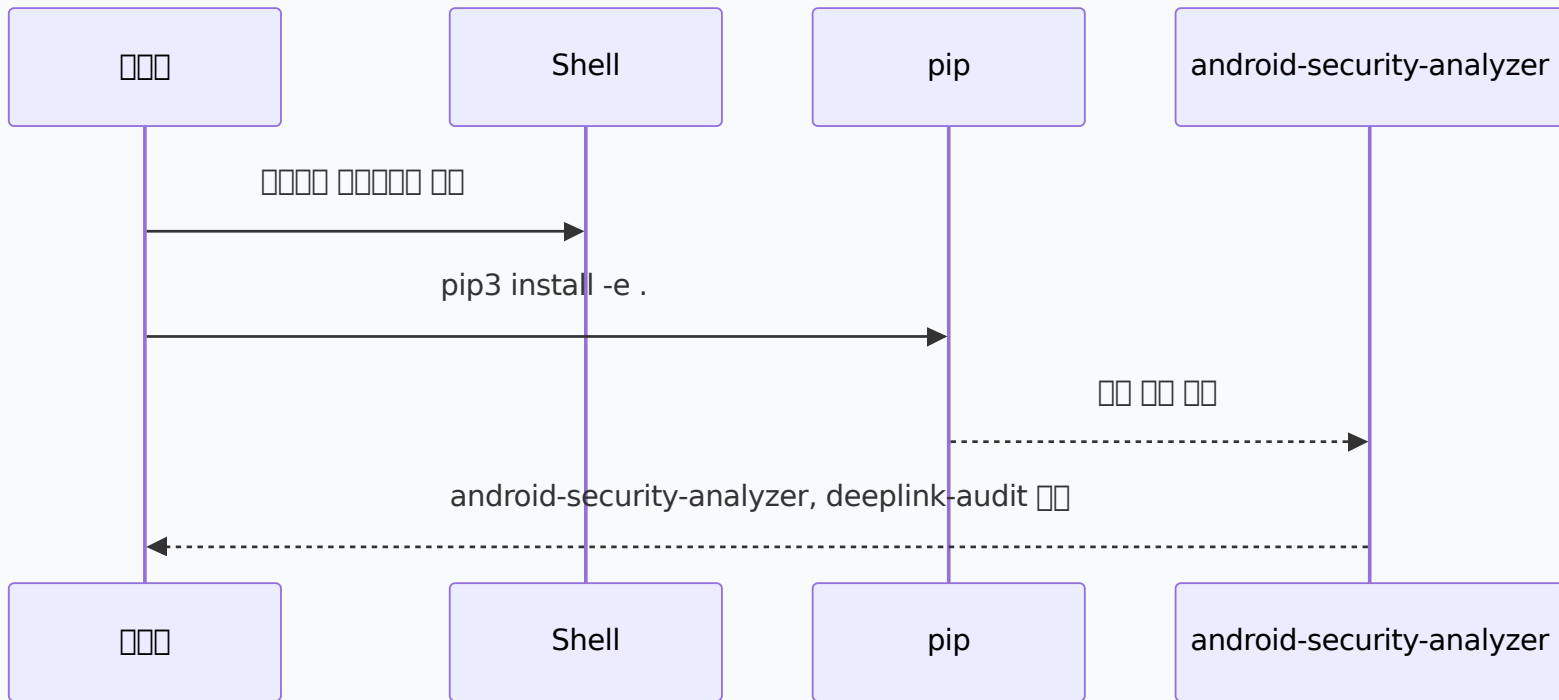


전체 구조 한눈에 보기



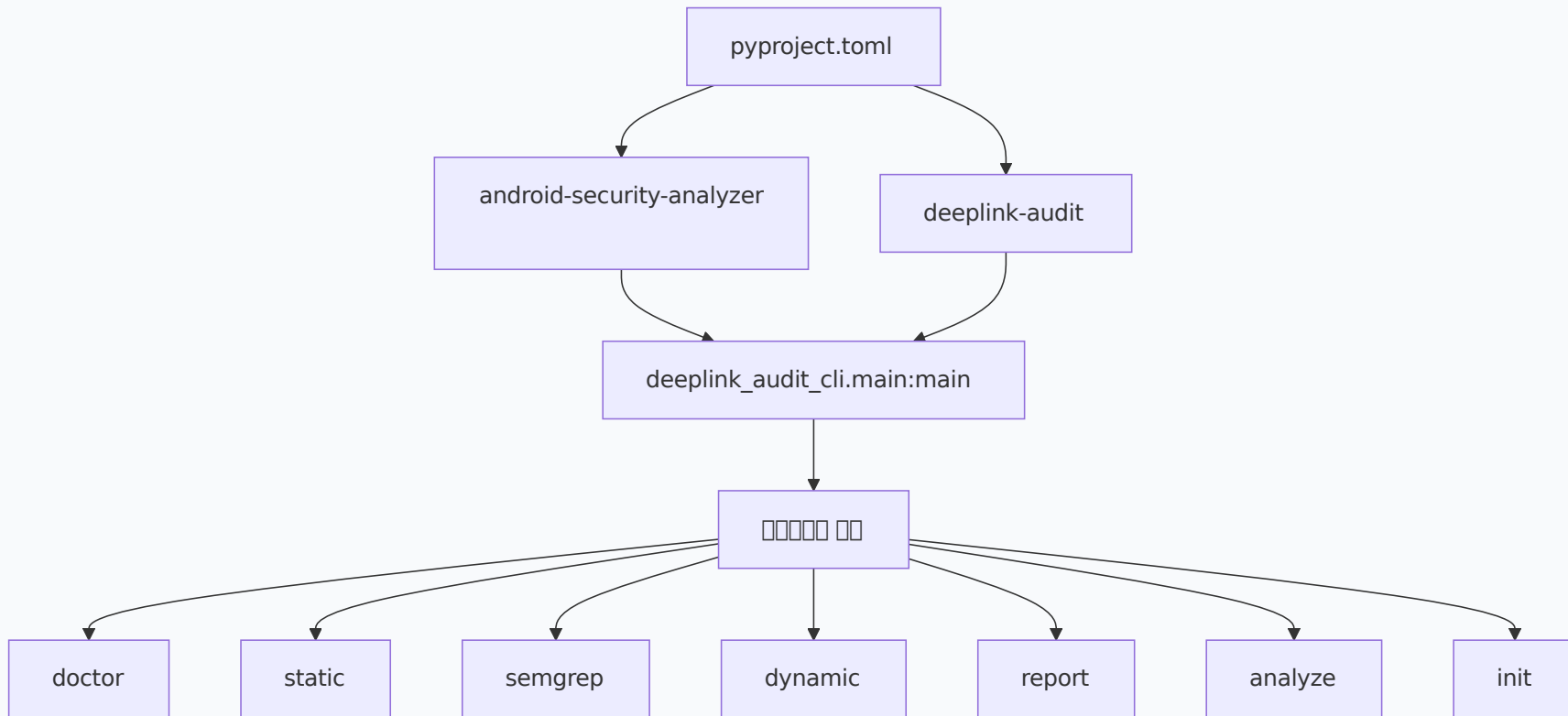
- 핵심은 Codex가 직접 분석 엔진이 아니라, 설치된 Skill과 CLI를 조율한다는 점이다.
- Static만으로 끝내지 않고 Dynamic까지 수행해 evidence를 확보한다.

설치 단계

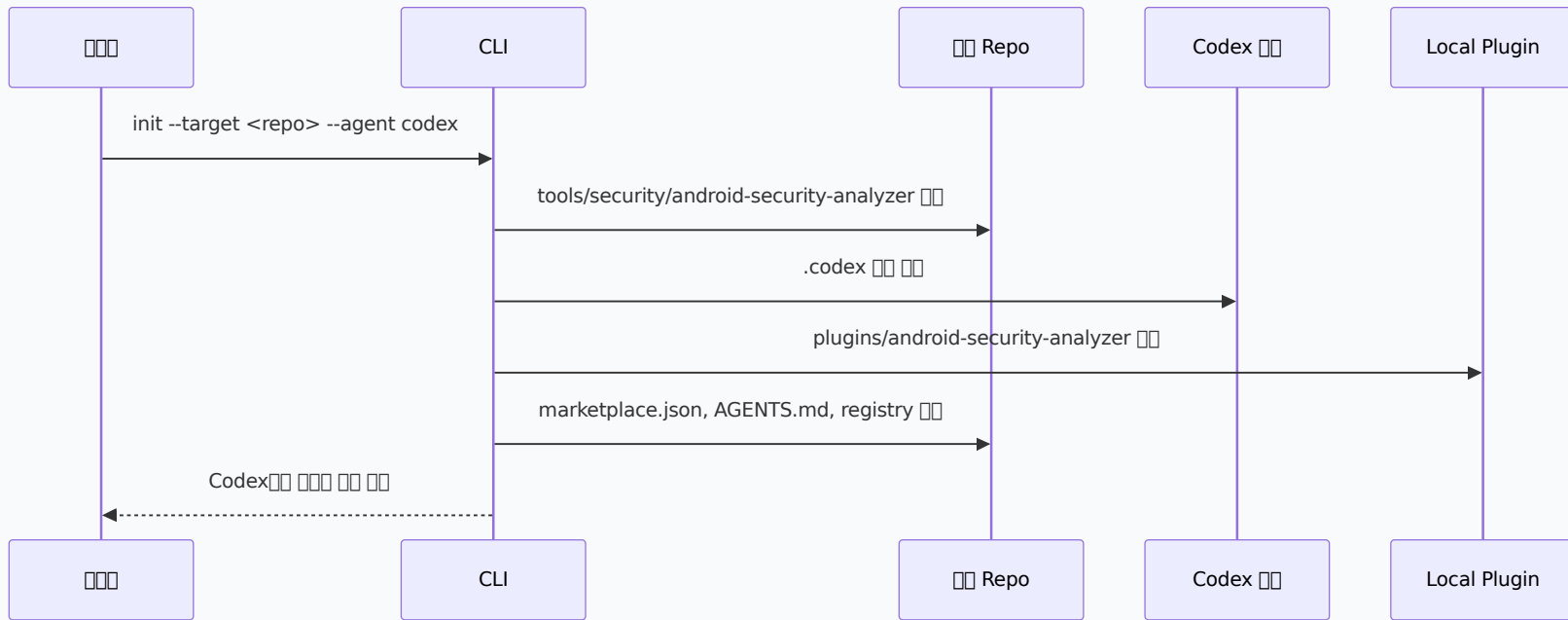


```
pip3 install -e .
```

엔트리포인트 구조



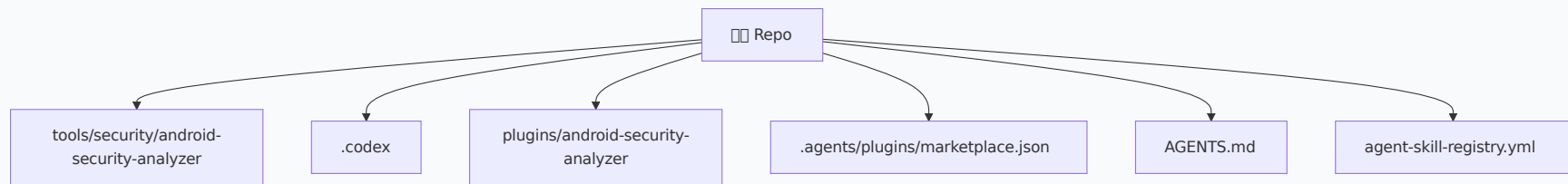
init 명령의 역할



■

`init` 은 분석을 수행하는 명령이 아니라, 분석을 위한 실행 환경을 repo에 심는 단계다.

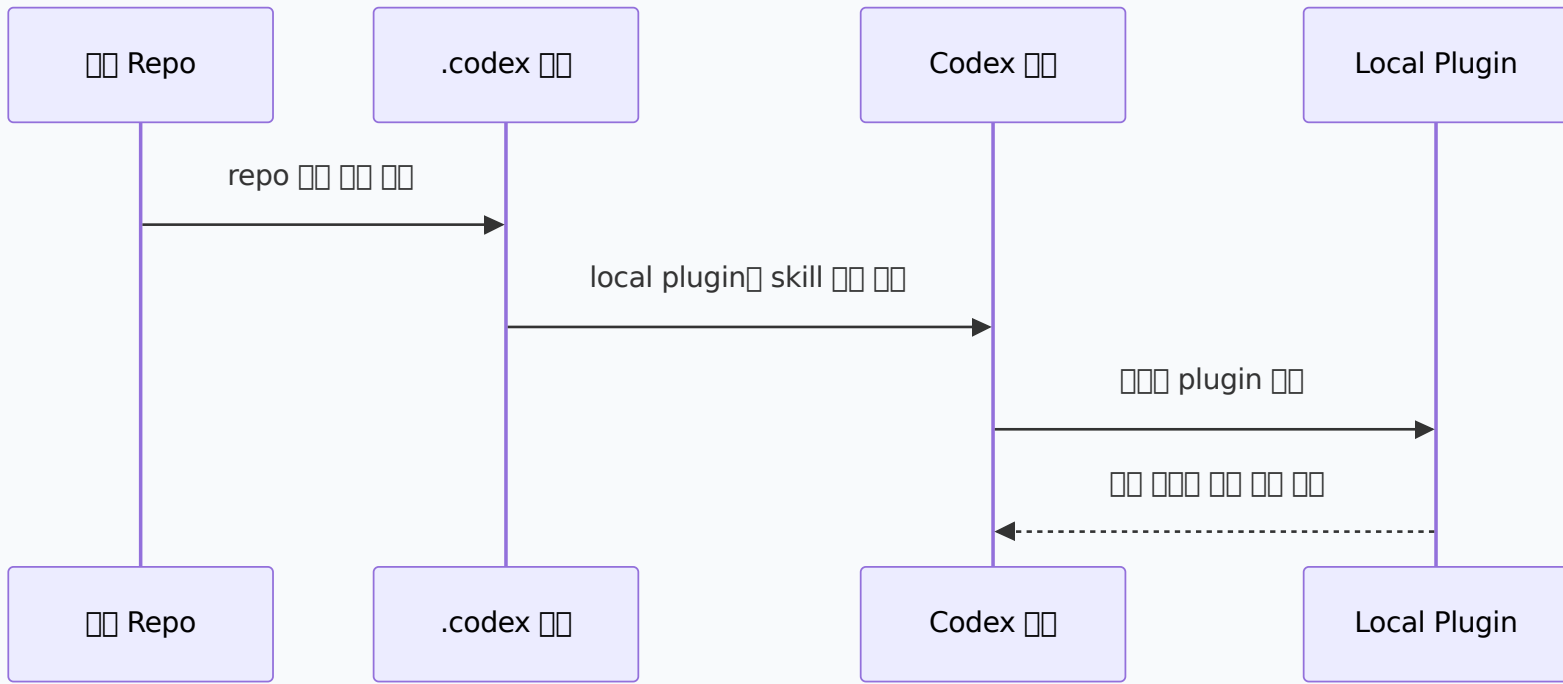
대상 repo에 생기는 구조



tools/security/android-security-analyzer/
 .codex/
 plugins/android-security-analyzer/
 .agents/plugins/marketplace.json
 AGENTS.md
 agent-skill-registry.yml

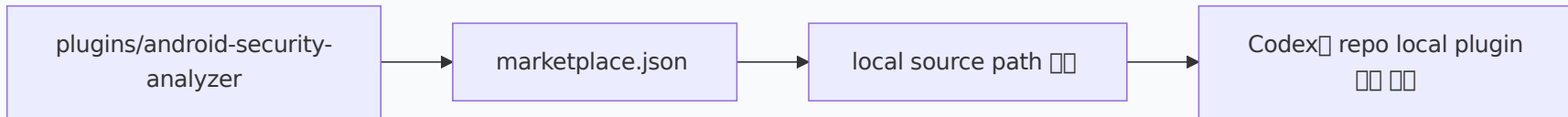
tools 폴더의 실제 역할

Codex 설정은 왜 필요한가



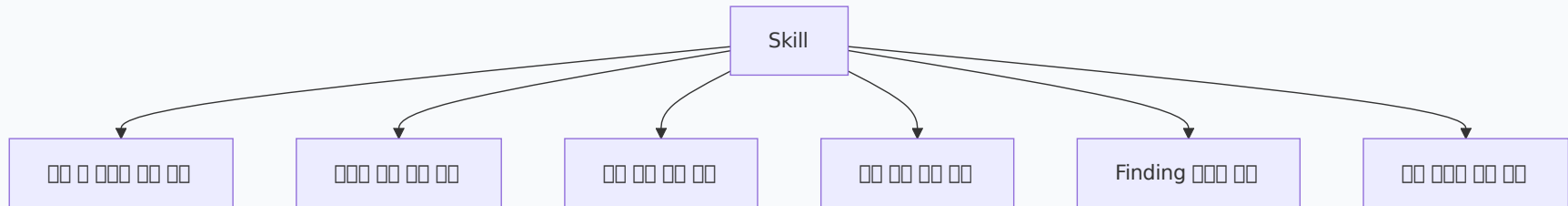
■ Codex는 이 설정을 보고 repo별 도구를 인식한다.

local plugin 등록 방식



- plugin은 “분석 기능이 여기 있다”는 연결점이다.
- 실제 실행 엔진은 여전히 CLI 쪽에 있다.

Skill이 하는 일

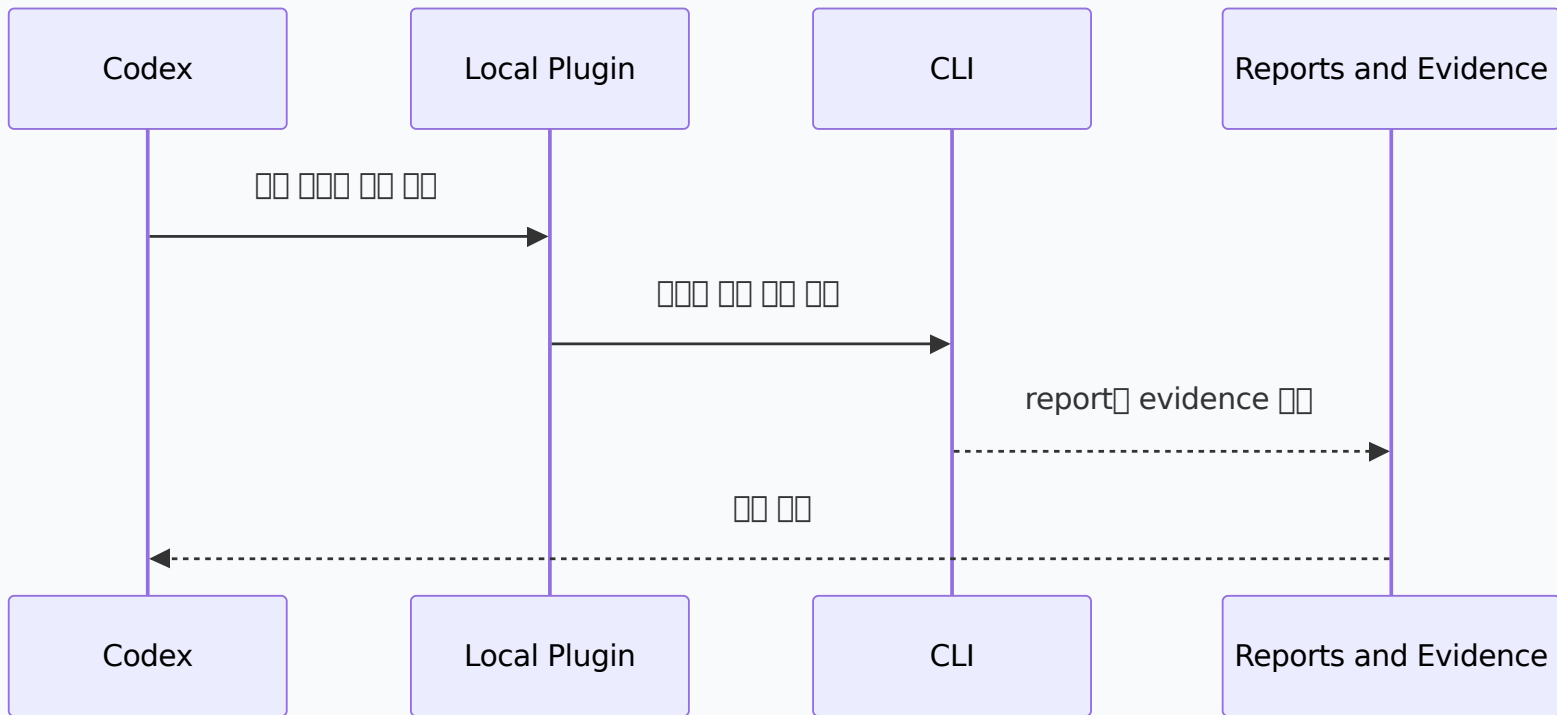


구체적으로 Skill에 들어가야 할 내용

- 대상 입력값 형식
- static에서 확인할 항목
- dynamic에서 반드시 검증할 항목
- evidence 저장 규칙
- finding 제목과 severity 형식
- MASVS나 내부 기준 매핑 방식

핵심 메시지

plugin이 하는 일



plugin은 Codex와 CLI 사이의 연결점이다.

실제 분석 엔진은 CLI다



각 단계 설명

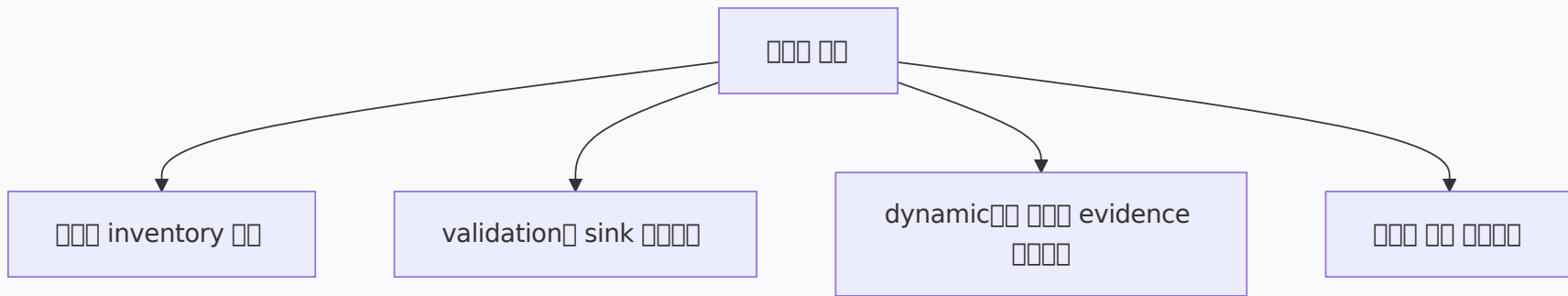
- doctor : 도구와 환경 준비 상태 확인
- static : manifest와 소스에서 딥링크 엔트리 추출
- semgrep : validation, sink, 위험 패턴 탐지
- dynamic : 실제 호출로 증거 확보, 결과 검증
- report : evidence를 묶어 finding과 보고서 생성

중요

- 여기서는 dynamic을 선택 기능이 아니라 필수 evidence 확보 단계로 본다.

Codex가 실제로 활용하는 흐름

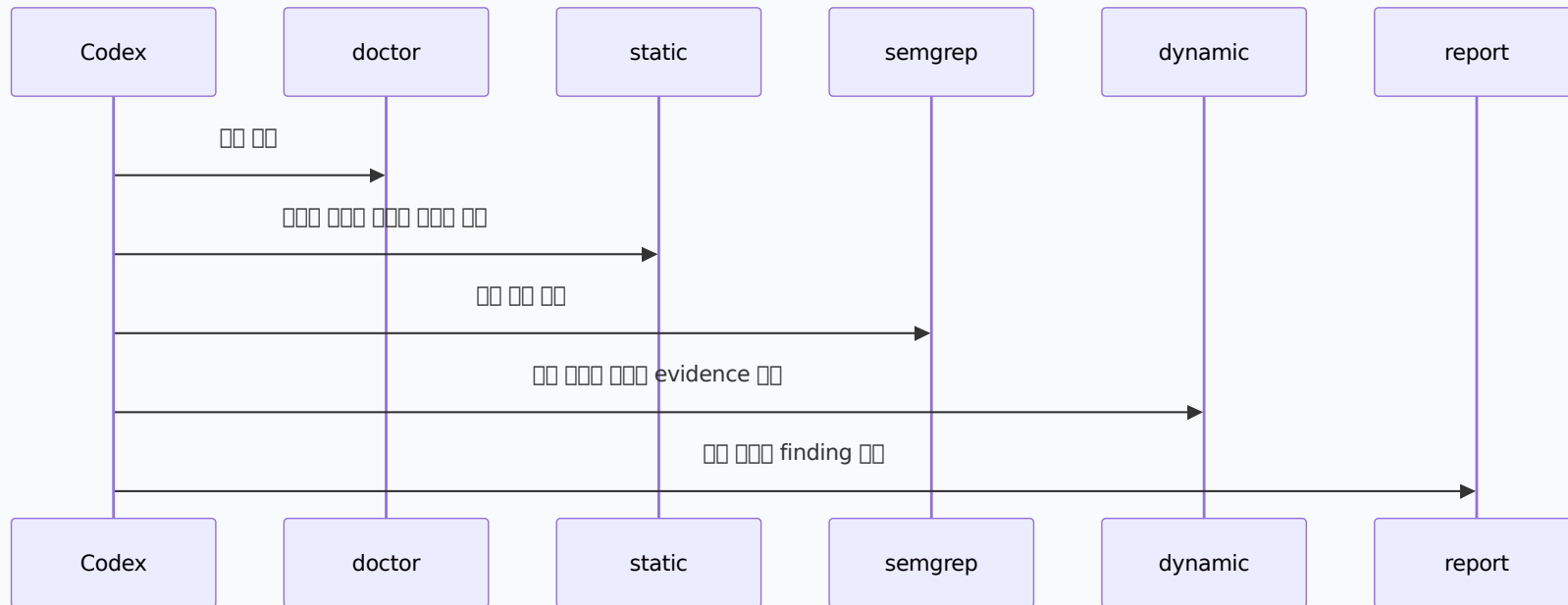
사용자는 어떻게 요청하면 좋은가



권장 요청 방식

- 대상 앱과 범위를 먼저 명시
- dynamic 검증까지 포함한다고 분명히 말하기
- 최종 산출물 형식도 같이 지정하기

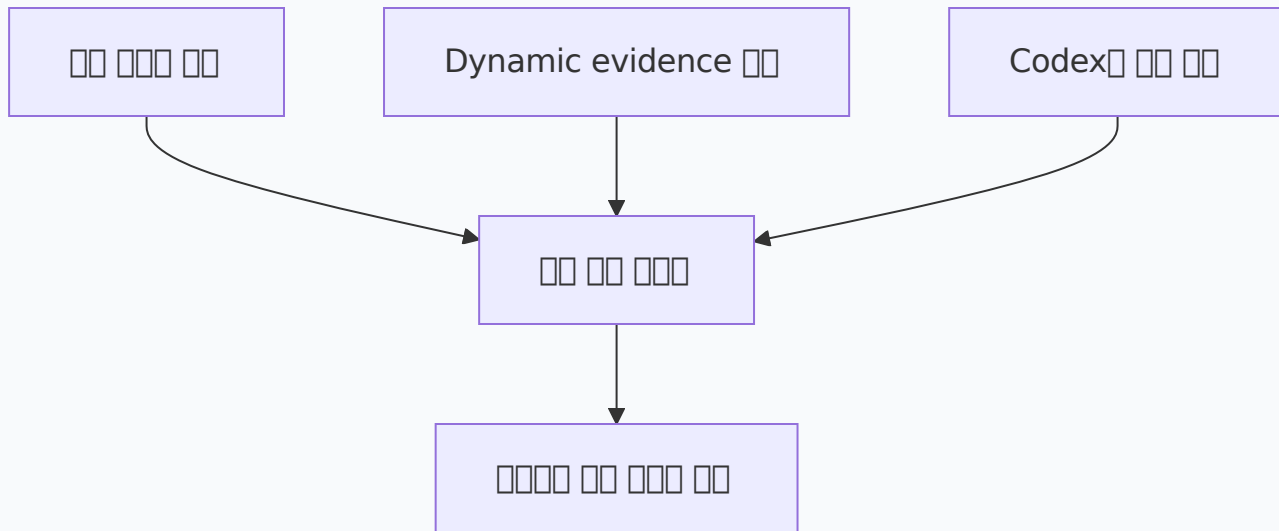
권장 실행 흐름



이 흐름을 권장하는 이유

주요 산출물과 확인 포인트

사용자가 얻는 실제 가치



■ 단순 자동화가 아니라, 증거 기반 감사 흐름을 반복 가능하게 만든다.

CLI만 쓸 때와 Codex를 붙였을 때

결론

